



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Programación orientada a objetos.

Laboratorio No. 6: Relaciones (Asociación y Composición)

Alumno:

Joshua Osorio O. – 1293271

Docente:

Gustavo O. Zamarron De La Garza

Septiembre/2026

Tabla de contenido

I.	Objetivo	3
II.	Descripción de la practica.....	3
	Lista de materiales.....	3
III.	Desarrollo de la práctica	4
	Metodología y Arquitectura	4
	Descripción del problema y elementos clave	4
	Datos de entrada y datos de salida	4
	Diagrama de clases.....	5
	Fragmentos de código y visualización de elementos clave	7
	Clase Mascota	7
	Clase Doctor	7
	Clase Veterinaria.....	8
	Clase Main — menú principal y control de sesión	9
	Reflexión y análisis crítico	10
	Dificultades técnicas.....	10
	Fracasos, fallas, bugs y errores	11
	Cómo se resolvió.....	11
	Conclusión	11
IV.	Bibliografía.....	11

I. Objetivo

El alumno diseñará clases que modelen entidades del mundo real y las relacione entre sí mediante asociación y composición, respetando los principios de la orientación a objetos y las capacidades del lenguaje Java. En esta práctica se implementó un sistema de gestión de consultas de una veterinaria, en el que una clase Veterinaria administra a sus doctores y a sus mascotas, controla el acceso mediante un inicio de sesión con dos tipos de usuario (super usuario y doctor), y una clase Main independiente expone toda esa funcionalidad a través de un menú interactivo.

II. Descripción de la practica

Se pide construir un sistema de gestión de consultas para una veterinaria, donde la veterinaria no es la clase principal. La veterinaria mantiene una lista de hasta 10 doctores y capacidad para registrar hasta 50 mascotas. Puede contratar, despedir y suspender doctores; un doctor suspendido sigue contratado, pero no puede realizar acciones dentro del sistema. Para usar la veterinaria un doctor debe iniciar sesión, y una vez dentro puede dar de alta una mascota, darla de baja y registrarle un diagnóstico, siempre que no esté suspendido. Además, debe existir un super usuario, que es el único autorizado para registrar, despedir y suspender doctores. Los doctores manejan información sensible (personal) e información de trabajo (usuario, clave y número de empleado), mientras que las mascotas guardan información sensible propia y de su dueño, además de su diagnóstico. Finalmente se pide un programa que demuestre todos los puntos anteriores registrando 3 doctores y 4 mascotas, y probando cada una de las funcionalidades.

Lista de materiales

- Computadora Personal (PC)
- Ambiente de desarrollo. (Puro bloc de notas papito)
- JDK (Java Development Kit).

III. Desarrollo de la práctica

Metodología y Arquitectura

Descripción del problema y elementos clave

A diferencia de prácticas anteriores, aquí el enunciado no dictaba una lista cerrada de clases con sus atributos, sino un conjunto de reglas de negocio: quién puede hacer qué, bajo qué condiciones y con qué límites. El trabajo de análisis consistió entonces en decidir cuántas clases hacían falta, qué información guardaba cada una y, sobre todo, qué tipo de relación existía entre ellas. Terminé con cuatro clases: Mascota y Doctor como entidades de datos, Veterinaria como la clase que concentra las reglas del negocio y el control de acceso, y Main como punto de entrada con el menú, ya que el enunciado pedía explícitamente que la veterinaria no fuera el main.

Los elementos clave del diseño son los siguientes:

- La Veterinaria mantiene a sus doctores y a sus mascotas en arreglos de tamaño fijo (Doctor[10] y Mascota[50]) con su respectivo contador, y es ella quien los crea al contratar un doctor o al dar de alta una mascota. Esa relación es de composición: si la veterinaria deja de existir, sus doctores y sus mascotas dejan de existir con ella.
- El Doctor guarda además un arreglo mascotasAtendidas con las mascotas a las que ya les dio un diagnóstico. Esta relación es de asociación y no de composición, porque el doctor solamente conserva una referencia a mascotas que la Veterinaria ya había creado: no las construye ni controla su ciclo de vida. Hay que mencionar que para casos prácticos este arreglo es más sencillo de manejar que utilizar un identificador, ya que no se utilizara datos externos y todo vive en memoria.
- El control de acceso vive completo dentro de la Veterinaria y no en el menú: cada operación verifica primero si hay sesión de super usuario o de doctor, y las tres acciones sobre mascotas comparten un método privado puedeOperarComoDoctor() que rechaza la operación si no hay doctor logueado o si el doctor está suspendido. La información sensible se protegió con encapsulamiento; por ejemplo, la clave del doctor no tiene getter y solo puede comprobarse con verificarClave().

Datos de entrada y datos de salida

Datos de entrada

- Menú principal: número entero para elegir la opción (1 y 2 iniciar sesión, 3 cerrar sesión, 4 a 6 gestión de doctores, 7 a 9 gestión de mascotas, 10 y 11 consultas, 12 datos de prueba, 0 salir).
- Inicio de sesión: usuario y clave como cadenas, tanto para el super usuario como para el doctor.
- Contratación de un doctor: nombre y teléfono (información personal), más usuario y clave (información de trabajo). El número de empleado no se captura, lo asigna la veterinaria.
- Alta de una mascota: nombre, especie y edad de la mascota, más nombre y teléfono del dueño; y para dar de baja o diagnosticar, el id de la mascota y el texto del diagnóstico.

Datos de salida

- Mensajes de confirmación con el identificador que asignó el sistema (por ejemplo, "Doctor contratado con éxito. Número de empleado: 1" o "Mascota registrada con éxito. Id: 1").
- Listados de doctores y de mascotas generados con el `toString()` de cada clase, alineados en columnas con `String.format()` para que puedan leerse como una tabla.
- Mensajes de error de validación cuando la operación no procede: sin sesión iniciada, con el doctor suspendido, con un usuario repetido, con campos obligatorios vacíos o con un id que no existe.

Diagrama de clases

El siguiente diagrama muestra las cuatro clases del sistema y el tipo de relación que existe entre ellas. Las líneas con diamante relleno que salen de Veterinaria hacia Doctor y hacia Mascota representan composición, porque la veterinaria crea esos objetos y controla su ciclo de vida (una veterinaria con 0 a 10 doctores y con 0 a 50 mascotas). La línea simple entre Doctor y Mascota representa asociación: el doctor guarda referencias a las mascotas que ha atendido, pero no es dueño de ellas. Main aparece unido a Veterinaria mediante una relación de uso, ya que únicamente la instancia para exponer sus operaciones en el menú.

UABC_FCQI_PROGRAMACIÓN_ORIENTADA_A_OBJETOS.

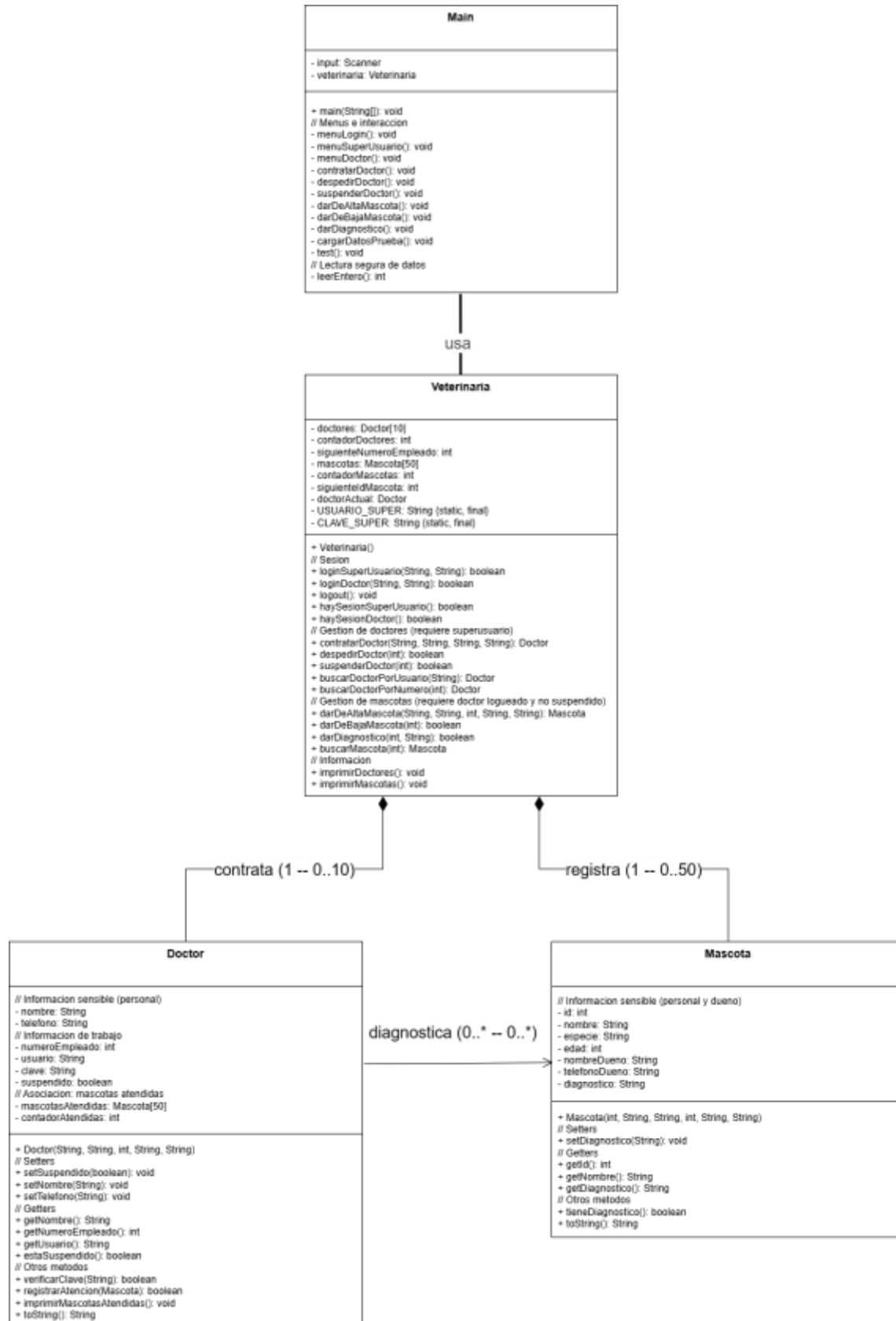


Ilustración 1: Diagrama de clases — Main, Veterinaria, Doctor y Mascota.

Fragmentos de código y visualización de elementos clave

Clase Mascota

```

/* ***** Otros metodos ***** */
public boolean tieneDiagnostico() {
    return diagnostico != null && !diagnostico.isEmpty();
}

@Override
public String toString() {
    String diag = tieneDiagnostico() ? diagnostico : "Sin diagnostico";
    return String.format("[Mascota %-2d]\t%-10s (%-8s, %2d años) | Dueño: %-17s, tel. %s\n",
        id, nombre, especie, edad, nombreDueno, telefonoDueno, diag);
}
}

```

Mascota es una clase de datos: guarda la información sensible de la mascota y la de su dueño, más el diagnóstico, que arranca en null porque una mascota recién registrada todavía no ha sido revisada. Por eso agregué tieneDiagnostico(), que comprueba null y cadena vacía a la vez, en lugar de dejar que cada clase que la use repita esa comprobación por su cuenta. En el toString() alineé cada campo con anchos fijos (%-10s, %-8s, %2d) para que al imprimir varias mascotas se lean como una tabla y no como texto corrido; el punto final lo dejé fuera del formato, porque el propio texto del diagnóstico ya trae su puntuación.

```

----- 6) Dar de alta mascotas (doctor Yuls logueada) -----
Doctor actual : [Doctor #1] Yuliana Flores | usuario: Yuls | tel. +16641112233 | estado: activo
Registrada: [Mascota 1] Aruma (Perro, 3 años) | Dueño: Xcaret Palma, tel. 664-444-5566 | Diagnostico: Sin diagnostico
Registrada: [Mascota 2] Sky (Lobo, 2 años) | Dueño: Daniela Rosa, tel. 664-555-6677 | Diagnostico: Sin diagnostico
Registrada: [Mascota 3] Caguama (Conejo, 5 años) | Dueño: Hiromi Ortiz, tel. 664-444-5566 | Diagnostico: Sin diagnostico
Registrada: [Mascota 4] Joy (Gato, 1 años) | Dueño: Samantha Andrade, tel. 664-666-7788 | Diagnostico: Sin diagnostico

Dar de alta con nombre vacío (inválido, debe rechazar):
Error: el nombre de la mascota y el del dueño son obligatorios.

```

Ilustración 2: Alta de las cuatro mascotas y salida del toString() de Mascota

Clase Doctor

```

/* ***** Otros metodos ***** */
/* No hay getClave(): la clave es información sensible, solo se verifica. */
public boolean verificarClave(String intento){return clave.equals(intento);}
public boolean estaSuspendido(){return suspendido;}

```

En Doctor separé la información en dos bloques: la personal (nombre y teléfono) y la de trabajo (número de empleado, usuario, clave y estado de suspensión). La clave es el dato más delicado, así que decidí no darle getter: la única forma de usarla desde fuera es verificarClave(), que recibe el intento y responde verdadero o falso, de manera que la contraseña nunca sale del objeto. estaSuspendido() funciona como candado y lo consulta la Veterinaria antes de permitir cualquier operación. En este mismo objeto vive el arreglo mascotasAtendidas, que es donde queda registrada la asociación con Mascota.

```

===== TEST: todas las operaciones =====

----- 1) Login de super usuario -----
Error: usuario o clave de administrador incorrectos.
Credenciales invalidas (debe rechazar): false
Credenciales validas (debe aceptar): true

----- 2) Contratar doctores -----
Contratado: [Doctor #1] Yuliana Flores | usuario: Yuls | tel. +16641112233 | estado: activo
Contratado: [Doctor #2] Zaide Quintero | usuario: Pato | tel. +526642223344 | estado: activo
Contratado: [Doctor #3] Andrea Mercado | usuario: Andy | tel. +526643334455 | estado: activo

Contratar con usuario repetido (invalido, debe rechazar):
Error: ya existe un doctor con ese usuario.

Contratar con nombre vacio (invalido, debe rechazar):
Error: nombre, usuario y clave son obligatorios.

----- 3) Sin sesion de super usuario no se puede contratar -----
Error: se necesita sesion de administrador para contratar doctores.
Intentar contratar sin login (invalido, debe rechazar): true

----- 4) Login de doctores -----
Error: usuario o clave de doctor incorrectos.
Clave incorrecta (invalido, debe rechazar): false
Credenciales validas (debe aceptar): true

```

Ilustración 3: Contratación de doctores y validación de credenciales de acceso

Clase Veterinaria

```

public boolean haySesionDoctor() {return doctorActual != null;}

public boolean darDiagnostico(int id, String diagnostico) {
    if (!puedeOperarComoDoctor()) {return false;}
    if (diagnostico == null || diagnostico.isEmpty()) {
        System.out.println("Error: el diagnostico no puede estar vacio.");
        return false;
    }
    Mascota mascota = buscarMascota(id);
    if (mascota == null) {
        System.out.println("Error: no existe una mascota con ese id.");
        return false;
    }

    mascota.setDiagnostico(diagnostico);
    doctorActual.registrarAtencion(mascota);
    return true;
}

```

Estas líneas del método darDiagnostico() son el centro de la práctica, porque en ellas conviven los dos tipos de relación. La primera modifica una Mascota que la propia Veterinaria creó y tiene guardada en su arreglo, que es la parte de composición. La segunda le avisa al doctor que tiene la sesión abierta para que anote esa misma mascota en su historial, y ahí aparece la asociación: el doctor termina apuntando al mismo objeto Mascota sin haberlo creado y sin poder destruirlo. Preferí guardar el objeto y no solamente su id, porque así el doctor puede consultar su historial por su cuenta sin depender de la veterinaria para resolver cada id.

```

=====
VETERINARIA - SISTEMA DE GESTION DE CONSULTAS
=====
(Modo demo) Super usuario -> usuario: root  clave: admin

[Sin sesion iniciada]

1 - Iniciar sesion como super usuario
2 - Iniciar sesion como doctor
3 - Cerrar sesion
4 - Contratar doctor
5 - Despedir doctor
6 - Suspender / reactivar doctor
7 - Dar de alta mascota
8 - Dar de baja mascota
9 - Dar diagnostico a una mascota
10 - Mostrar doctores
11 - Mostrar mascotas
12 - Cargar datos de prueba
0 - Salir del programa

Opcion: 1

----- Iniciar sesion como super usuario -----
Usuario:      root
Clave: admin
Sesion de super usuario iniciada.

[Session activa: SUPER USUARIO]

1 - Iniciar sesion como super usuario
2 - Iniciar sesion como doctor
3 - Cerrar sesion
4 - Contratar doctor
5 - Despedir doctor
6 - Suspender / reactivar doctor
7 - Dar de alta mascota
8 - Dar de baja mascota
9 - Dar diagnostico a una mascota
10 - Mostrar doctores
11 - Mostrar mascotas
12 - Cargar datos de prueba
0 - Salir del programa

Opcion: |

```

Ilustración 4: Inicio de sesión validado por la Veterinaria antes de permitir cualquier operación

Clase Main — menú principal y control de sesión

```

public static void main(String[] args){
    int opcion;
    do{
        imprimirEstadoSesion();
        // ...imprime el menú principal...
        opcion = leerEntero();
        switch(opcion){
            case 1: loginSuperUsuario(); break;
            case 2: loginDoctor();      break;
            case 7: darDeAltaMascota();  break;
            case 9: darDiagnostico();    break;
            case 0: System.out.println("Saliendo del programa..."); break;
            default: System.out.println("Opcion no valida, intente de nuevo.");
        }
    } while(opcion != 0);
}

```

UABC_FCQI_PROGRAMACIÓN_ORIENTADA_A_OBJETOS.

Main es únicamente la capa de interacción: no guarda doctores ni mascotas, solamente un Scanner y un objeto Veterinaria, y cada opción del menú se limita a pedir los datos y delegarlos. Preferí un solo menú con el estado de la sesión impreso arriba, en vez de un submenú por rol, porque las mismas opciones cambian de comportamiento según quién esté logueado y es la Veterinaria la que decide si la operación procede o no. La lectura de enteros la aislé en leerEntero(), que atrapa InputMismatchException y vuelve a pedir el dato, para que escribir una letra en el menú no tumbe el programa. También dejé una opción oculta, la 13, que no se imprime en el menú y ejecuta una prueba automática de todas las operaciones, válidas e inválidas, sobre una veterinaria aparte; es la que usé para verificar la práctica y de donde salieron varias de las capturas de este reporte.

```
~\OneDrive - uabc.edu.mx\9  X  +  v
PowerShell 7.6.6
Página de códigos activa: 65001
La carga de perfiles personales y del sistema tardó 768ms.
> cd "C:\Users\Okuyt\OneDrive - uabc.edu.mx\9_Semestre\36282_Programacion_Orientada_a_Objeto\output\"
> java Main
=====
VETERINARIA - SISTEMA DE GESTION DE CONSULTAS
=====
(Modo demo) Super usuario -> usuario: root  clave: admin

[Sin sesion iniciada]

1 - Iniciar sesion como super usuario
2 - Iniciar sesion como doctor
3 - Cerrar sesion
4 - Contratar doctor
5 - Despedir doctor
6 - Suspender / reactivar doctor
7 - Dar de alta mascota
8 - Dar de baja mascota
9 - Dar diagnostico a una mascota
10 - Mostrar doctores
11 - Mostrar mascotas
12 - Cargar datos de prueba
0 - Salir del programa

Opcion: |
```

Ilustración 5: Menú principal del sistema de gestión de consultas

Reflexión y análisis crítico

Dificultades técnicas

Lo que más trabajo me costó no fue escribir el código sino decidir cómo relacionar al Doctor con la Mascota. Mi primera idea fue que el doctor guardara solo los id de las mascotas que atendió y pedirle a la Veterinaria que los resolviera cada vez, porque me parecía más ligero. Al analizarlo me di cuenta de que eso obligaba al Doctor a depender de la Veterinaria para consultar su propio historial, cuando la relación que pide la práctica es directa entre Doctor y Mascota. También consideré crear una clase Consulta que representara la atención en sí, con doctor, mascota, fecha y diagnóstico, que es como se modelaría en un sistema real donde la relación guarda datos propios; la descarté porque para el alcance de esta práctica agregaba complejidad que el enunciado no pide. La otra dificultad fue el manejo de las dos sesiones: tuve que decidir dónde vivía la

validación, y opté por dejarla dentro de la Veterinaria y no en el menú, para que ninguna operación pudiera ejecutarse saltándose el control de acceso.

Fracasos, fallas, bugs y errores

El error más molesto fue una excepción en tiempo de ejecución: `java.util.IllegalFormatConversionException: d != java.lang.String`, que tumbaba el programa justo al imprimir la primera mascota. En el `String.format()` del `toString()` de `Mascota` se me había ido el signo de porcentaje del primer especificador: escribí `"#-5d"` en lugar de `"%-5d"`. Como `"#-5d"` no es un especificador válido, Java lo tomó como texto normal y no consumió ningún argumento, así que todos los especificadores siguientes se recorrieron un lugar y el `%d` que era para la edad terminó recibiendo la especie, que es un `String`. También tuve dos detalles menores: el diagnóstico salía con doble punto al final, porque el formato ya traía un punto fijo y mis textos de prueba también terminaban en punto; y los acentos y la ñ de "Dueño" aparecían como signos de interrogación en la consola.

Cómo se resolvió

La excepción la ubiqué leyendo la traza de la pila de abajo hacia arriba: la última línea que pertenecía a mi código era `Mascota.toString()`, así que el problema estaba ahí y no donde parecía. Agregando el signo de porcentaje que faltaba quedó resuelto, y de paso entendí que un especificador mal escrito no da error de compilación, solo recorre los argumentos y truena hasta que corres el programa. Y lo de los acentos no era un problema del código sino de la codificación de la consola: se corrigió compilando con `javac -encoding UTF-8` y poniendo la consola en UTF-8 con `chcp 65001` antes de ejecutar.

Conclusión

Esta práctica me sirvió para ver la diferencia real entre composición y asociación, que hasta ahora se me hacía una distinción más de diagrama que de código. Al implementarla queda claro que no se trata de cómo se dibuja la flecha, sino de quién crea el objeto y quién puede destruirlo: la Veterinaria crea y elimina doctores y mascotas, mientras que el Doctor solo apunta a mascotas que ya existían. También me quedó la idea de que el control de acceso conviene ponerlo en la clase que tiene las reglas y no en el menú, porque así la validación se cumple, aunque después se agregue otra forma de llamar a esos métodos.

IV. Bibliografía

- [1] Oracle Corporation. (2024). Java SE Documentation. Recuperado de <https://docs.oracle.com/javase/>
- [2] Deitel, P. J., & Deitel, H. M. (2016). Java: How to Program (10th ed.). Pearson Education.